

Algoritmo de Prim y análisis amortizado

CC4102 — Diseño y Análisis de Algoritmos
Tarea 1 · Semestre 2026-2

Integrante: Javier Andreu Díaz Parada
Profesores: Benjamín Bustos y Gonzalo Navarro

1. Introducción

Esta tarea compara dos implementaciones del algoritmo de Prim para obtener un árbol cobertor mínimo. Una usa una cola binomial y la otra una cola de Fibonacci. La hipótesis es que la cola binomial tendrá una implementación más simple y constantes más pequeñas, mientras que la cola de Fibonacci debería verse favorecida cuando Prim realiza muchas llamadas a `decreaseKey`, debido a su costo amortizado constante.

2. Desarrollo

El grafo se guarda como lista de adyacencia. Para asegurar conectividad, el generador agrega primero un árbol donde cada vértice $v > 0$ se conecta con un vértice uniforme en $[0, v-1]$. Luego agrega pares aleatorios distintos y no reflexivos hasta completar exactamente e aristas. Los pesos se generan uniformemente en $(0, 1]$ y cada corrida usa una semilla distinta.

Ambas colas mantienen un arreglo de punteros desde el identificador del vértice hacia su nodo. En la cola binomial, `decreaseKey` intercambia el costo y el vértice con el padre y ambos punteros del arreglo se actualizan después de cada intercambio. En la cola de Fibonacci la identidad del nodo no cambia, `decreaseKey` corta el nodo cuando viola el orden de heap y aplica cortes en cascada mediante la marca de los anteriores.

La cola inicial se construye mediante inserciones sucesivas. En una cola de Fibonacci cada inserción cuesta $O(1)$. En una cola binomial las inserciones producen enlaces equivalentes a los de un contador binario así durante n inserciones cada rango se enlaza $O(n/2^k)$ veces, y la suma total es $O(n)$.

3. Análisis amortizado de `decreaseKey`

Sea $t(H)$ el número de raíces y $m(H)$ el número de nodos marcados. Se utiliza

$$\Phi(H) = A(t(H) + 2m(H)), \quad A > 0.$$

El potencial no es negativo en todo estado y es cero en la cola vacía. El factor A permite contabilizar el costo constante de las primitivas y evita cancelar informalmente un término $O(c)$ contra $-c$ sin fijar las unidades.

Si una llamada no corta nodos, el trabajo y el cambio de potencial están acotados por una constante. Si realiza $c \geq 1$ cortes, cada corte añade una raíz. Los últimos $c - 1$ cortes eliminan marcas de ancestros y al terminar se añade a lo más una marca. Por ello,

$$\Delta t = c, \quad \Delta m \leq -(c - 1) + 1 = 2 - c,$$

$$\Delta \Phi \leq A(c + 2(2 - c)) = A(4 - c).$$

En un modelo de operaciones elementales existen constantes $a, b > 0$ tales que el costo real cumple $C \leq ac + b$. Tomando $A \geq a$, el costo amortizado satisface

$$\hat{C} = C + \Delta \Phi \leq (a - A)c + b + 4A \leq b + 4A = O(1).$$

El crédito acumulado al marcar un nodo paga su corte futuro. Esto entrega costo constante amortizado, aunque una llamada individual pueda recorrer una cascada larga.

4. Complejidad de Prim

Prim ejecuta v extracciones de mínimo y a lo más e disminuciones de clave. Con cola binomial ambas operaciones cuestan $O(\log v)$, dando $O(e \log v)$. Con cola de Fibonacci, `extractMin` cuesta $O(\log v)$ amortizado y `decreaseKey` cuesta $O(1)$ amortizado, dando $O(e + v \log v)$.

5. Metodología experimental

Cada configuración se ejecuta diez veces con grafos distintos. Ambos algoritmos reciben exactamente el mismo grafo en cada repetición. La generación queda fuera del intervalo medido y el orden de ejecución se alterna entre repeticiones. Se usa `std::chrono::steady_clock`.

Las series A y B miden tiempo total. Las series C y D acumulan el tiempo de todas las llamadas a `decreaseKey` y cuentan intercambios en la cola binomial y cortes en la cola de Fibonacci. El texto de la sección 6.3.2 llama A/B a los gráficos amortizados, pero se interpretan como C/D porque esas son las series definidas inmediatamente antes.

Los resultados se guardan en `results/raw_results.csv`; los promedios quedan en `results/summary.csv`. Las curvas teóricas se multiplican por la constante de mínimos cuadrados que ajusta una recta por el origen. Los gráficos comparables usan la misma escala vertical.

5.1. Configuraciones

Serie	Vértices	Aristas	Medición
A	$v = 2^{20}$	$e = 2^j$, $j \in \{20, 21, 22, 23, 24\}$	Tiempo total
B	$v = 2^i$, $i \in \{18, 19, 20, 21, 22\}$	$e = 2^{24}$	Tiempo total
C	$v = 2^{18}$	$e = 2^j$, $j \in \{18, 19, 20, 21, 22\}$	<code>decreaseKey</code>
D	$v = 2^i$, $i \in \{14, 15, 16, 17, 18\}$	$e = 2^{22}$	<code>decreaseKey</code>

Las tablas reportan promedios de diez repeticiones.

5.2. Ambiente

Recurso	Valor
CPU	AMD Ryzen 7 8845HS, 8 núcleos y 16 hilos
Caché L1	No informada en el reporte de esta ejecución
Caché L2 / L3	8192 KiB / 16384 KiB
RAM	16 440 479 744 bytes (15.31 GiB)
Sistema operativo	Microsoft Windows 11 Pro 10.0.26200, 64 bits
Compilador	MSVC 19.50.35730.0
Lenguaje y flags	C++20, Release (/O2 /Ob2 /DNDEBUG)

6. Memoria

Para $v=2^{15}$ y $e=2^{20}$, el ejecutable `prim_mst --memory` reporta el cálculo basado en `sizeof` de esta compilación. Debe compararse con `results/system_info.txt`. La lista de adyacencia considera dos entradas por arista; luego se suman los nodos de cada cola y los arreglos de costos, padres, pertenencia y punteros directos.

Componente	Memoria estimada
Lista de adyacencia	32.75 MiB
Arreglos auxiliares	0.66 MiB
Nodos binomiales	1.50 MiB
Nodos Fibonacci	2.00 MiB
Total con cola binomial	34.91 MiB
Total con cola Fibonacci	35.41 MiB

La estimación corresponde al almacenamiento lógico del grafo y una cola a la vez. No incorpora capacidad sobrante de los vectores, metadatos del asignador ni estructuras temporales para detectar aristas repetidas. Es una estimación, no una medición de memoria máxima residente. Los valores implican aproximadamente 16 bytes por entrada de adyacencia, 24 bytes por cabecera de lista y 48/64 bytes por nodo binomial/Fibonacci, respectivamente. Estos tamaños deben contrastarse con los sizeof del ejecutable.

Los totales representan aproximadamente 0.22% de la RAM disponible. Las corridas más grandes sí usan más memoria que esta estimación puntual, especialmente por las listas de adyacencia y las estructuras temporales del generador.

7. Resultados

La corrida produjo 400 mediciones, 20 configuraciones, 10 grafos y dos colas, ambas implementaciones obtuvieron pesos de MST coincidentes dentro de la tolerancia numérica y $v-1$ aristas.

7.1. Costo total

Serie	i	j	Binomial (ms)	Fibonacci (ms)
A	20	20	1810.14	1718.04
A	20	21	2160.32	2184.69
A	20	22	2753.80	3186.90
A	20	23	3420.89	3669.88
A	20	24	3798.47	4087.77
B	18	24	995.64	1152.63
B	19	24	1853.72	2109.18
B	20	24	3787.10	4130.18
B	21	24	7566.98	7537.41
B	22	24	14403.59	14235.35

En A, Fibonacci fue 5.1% más rápida solo en el primer punto. Binomial ganó en los otros cuatro. En B, binomial fue más rápida hasta $i=20$, mientras que Fibonacci obtuvo ventajas pequeñas en $i=21$ y $i=22$ (0.4% y 1.2%). Estas diferencias son descriptivas así que sin desviaciones estándar ni intervalos de confianza no es posible establecer si las ventajas de 0.4% y 1.2% son estadísticamente distinguibles del ruido. Los resultados no bastan para atribuir esos cruces a la diferencia asintótica.

Las constantes de ajuste de A fueron $1.472e-5$ para binomial y $1.093e-4$ para Fibonacci. En B fueron $1.793e-5$ y $1.236e-4$, respectivamente. La cota de Fibonacci sigue especialmente bien la forma de B. La cota binomial de B es menos ajustada, así con e fijo, aumenta el número efectivo de extracciones y disminuciones al crecer v , además de empeorar localidad de caché, mientras la cota superior $e \log v$ cambia poco y no pretende ser una predicción exacta.

Costo total · serie A

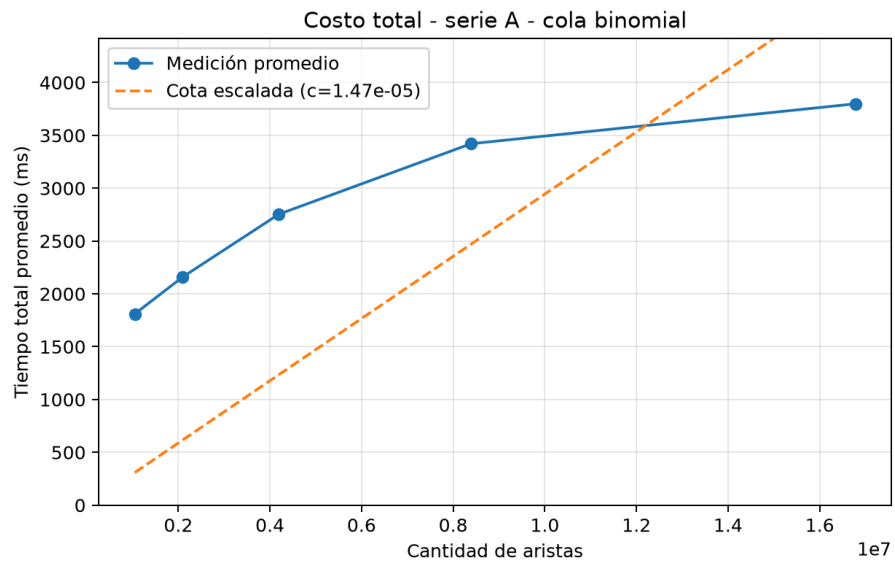


Figura 1: Costo total · serie A · cola binomial.

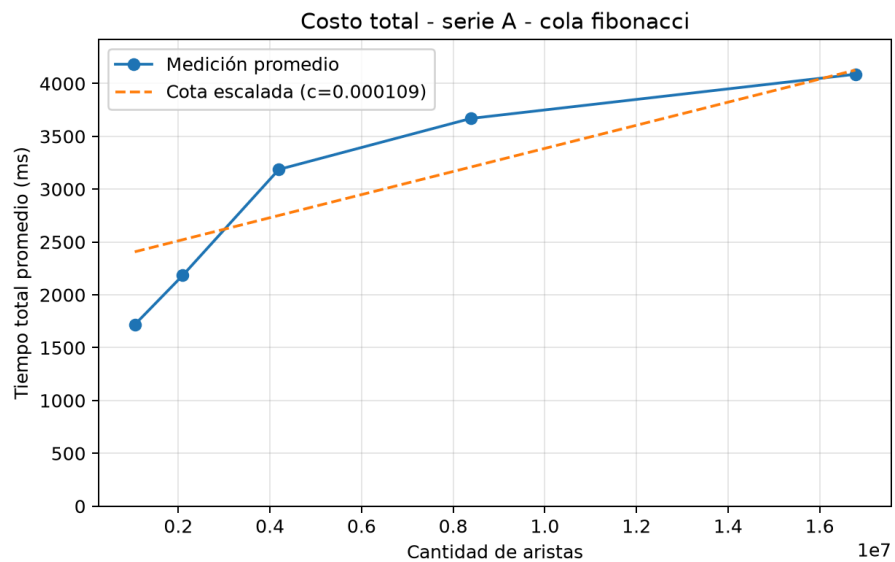


Figura 2: Costo total · serie A · cola de Fibonacci.

Costo total · serie B

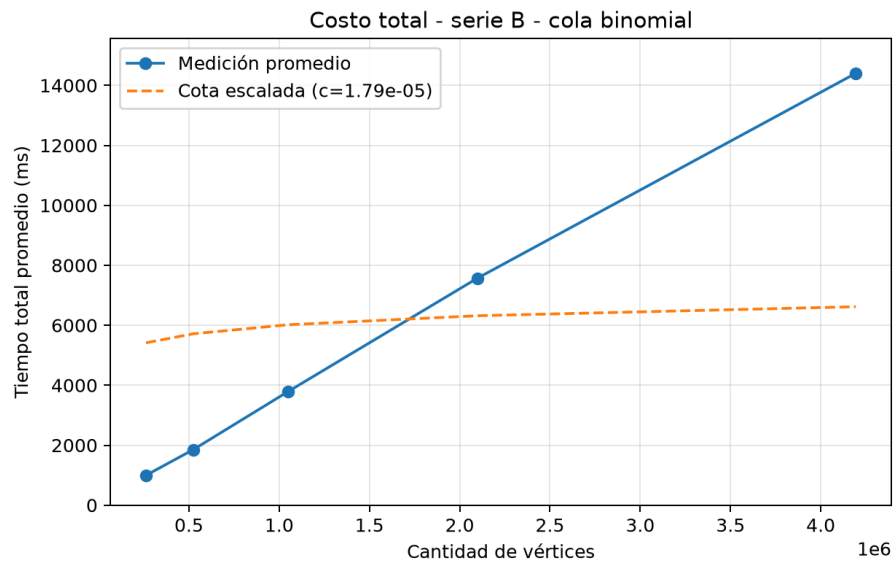


Figura 3: Costo total · serie B · cola binomial.

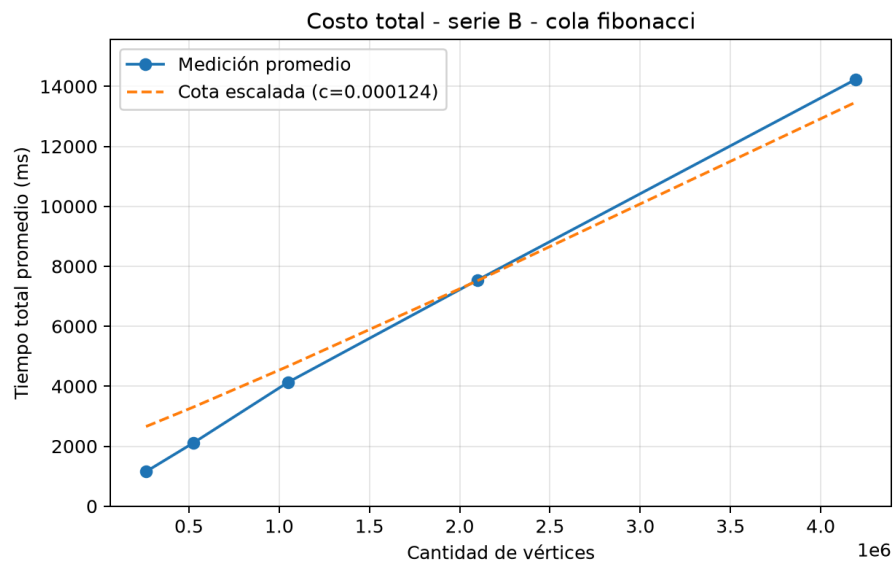


Figura 4: Costo total · serie B · cola de Fibonacci.

7.2. Costo amortizado

Serie	i	j	Llamadas	Bin. ms	Fib. ms	Intercambios	Cortes
C	18	18	262144	74.01	79.51	1762728	355174
C	18	19	426491	112.45	122.44	1609226	435543
C	18	20	604090	158.21	174.39	1696016	541536
C	18	21	785237	234.08	234.78	1789730	639139
C	18	22	966550	274.29	322.11	1882775	732309
D	14	22	105814	12.23	13.95	119918	67959
D	15	22	188873	25.64	29.09	237813	124945
D	16	22	332585	57.72	68.98	474335	227993
D	17	22	574208	130.38	147.68	946223	411532
D	18	22	966718	318.69	338.41	1884965	733015

Las formas son compatibles con cotas superiores amortizadas, pero no todas las curvas son lineales. En D, el tiempo por llamada aumenta al crecer v , para binomial pasa de aproximadamente 116 a 330 ns y para Fibonacci de 132 a 350 ns. El conteo de cortes por llamada permanece por debajo de uno en estos puntos. Esto distingue costo estructural amortizado de tiempo real por llamada. En C, v es fijo, por lo que $k \log v$ también es una recta respecto de k . En D el factor $\log v$ solo cambia de 14 a 18. Las constantes temporales ajustadas fueron $1.569e-5$ y $1.653e-5$ para binomial en C/D, y $3.112e-4$ y $3.108e-4$ para Fibonacci.

Fibonacci realizó muchos menos cambios estructurales en el último punto de C, 732309 cortes frente a 1882775 intercambios, y en el último de D, 733015 frente a 1884965. Sus constantes de conteo fueron 0.838 y 0.740. Las binomiales fueron 0.138 y 0.104 al escalar $k \log v$. Sin embargo, menos operaciones abstractas no significó menor tiempo, un corte modifica varios punteros en listas circulares, mientras un intercambio cambia contenidos de dos nodos. Aunque los nodos se almacenan contiguamente, las operaciones recorren enlaces mediante punteros. Los cortes de Fibonacci modifican más enlaces y las diferencias de localidad no fueron medidas directamente. También influye el costo fijo de invocar el reloj para cada llamada.

Serie C · tiempo acumulado

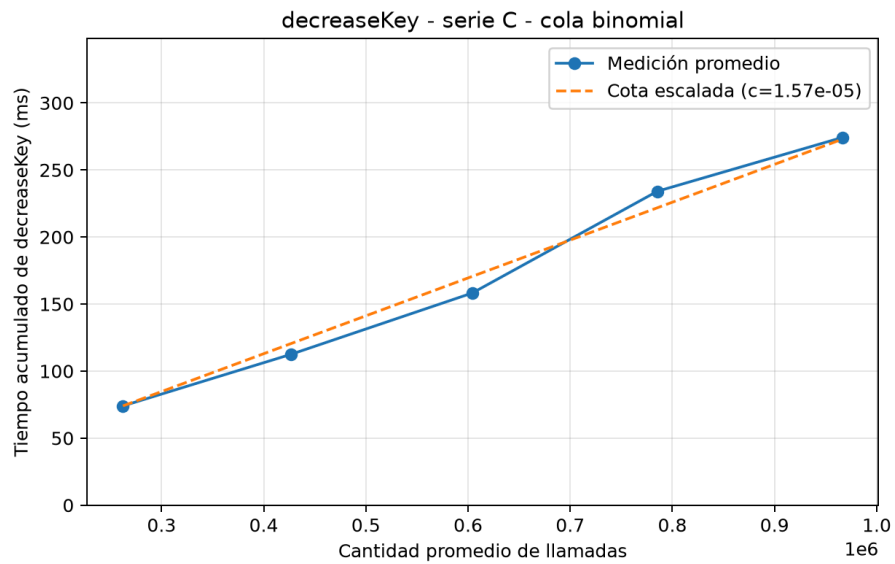


Figura 5: Serie C · tiempo acumulado · cola binomial.

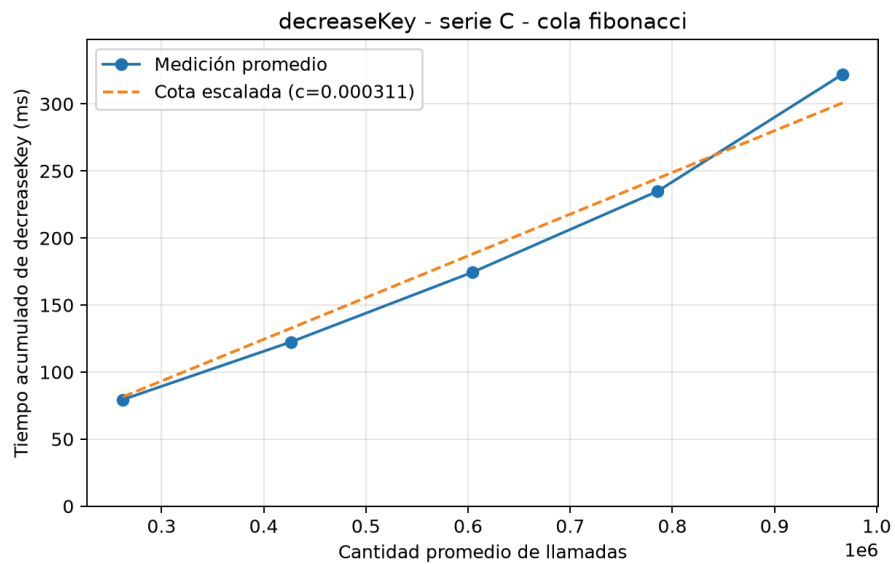


Figura 6: Serie C · tiempo acumulado · cola de Fibonacci.

Serie C · operaciones estructurales

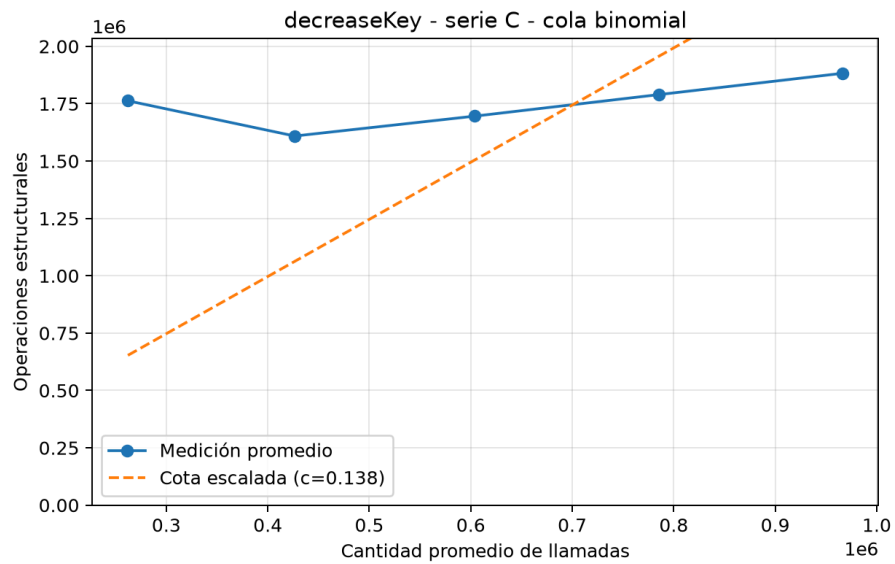


Figura 7: Serie C · operaciones estructurales · cola binomial.

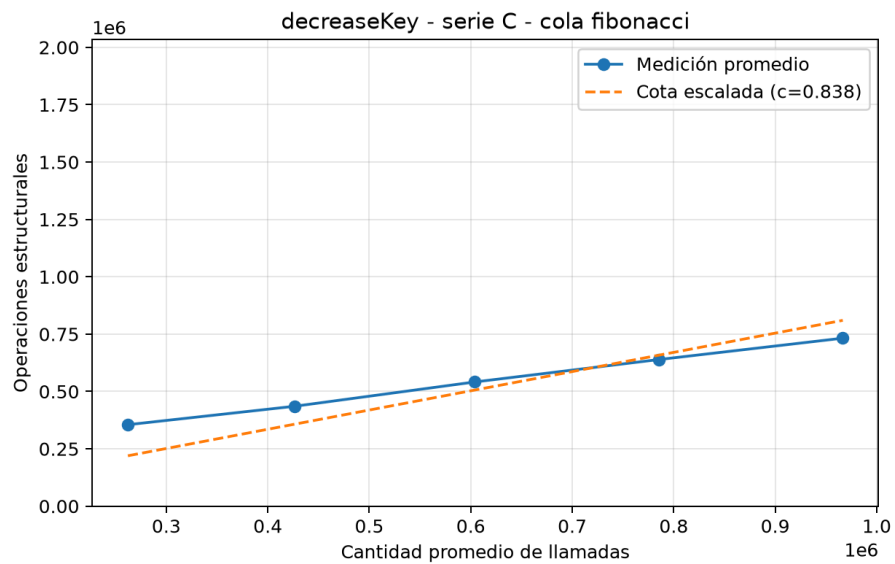


Figura 8: Serie C · operaciones estructurales · cola de Fibonacci.

Serie D · tiempo acumulado

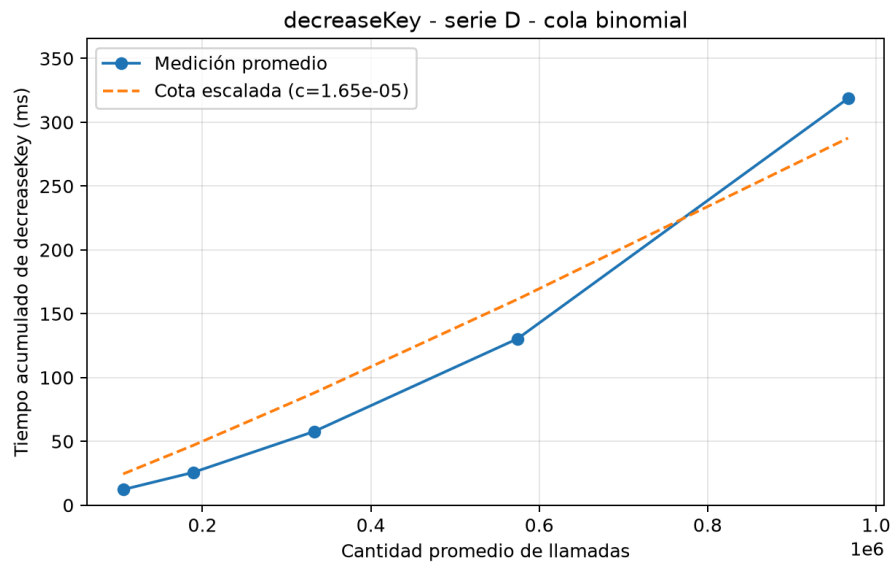


Figura 9: Serie D · tiempo acumulado · cola binomial.

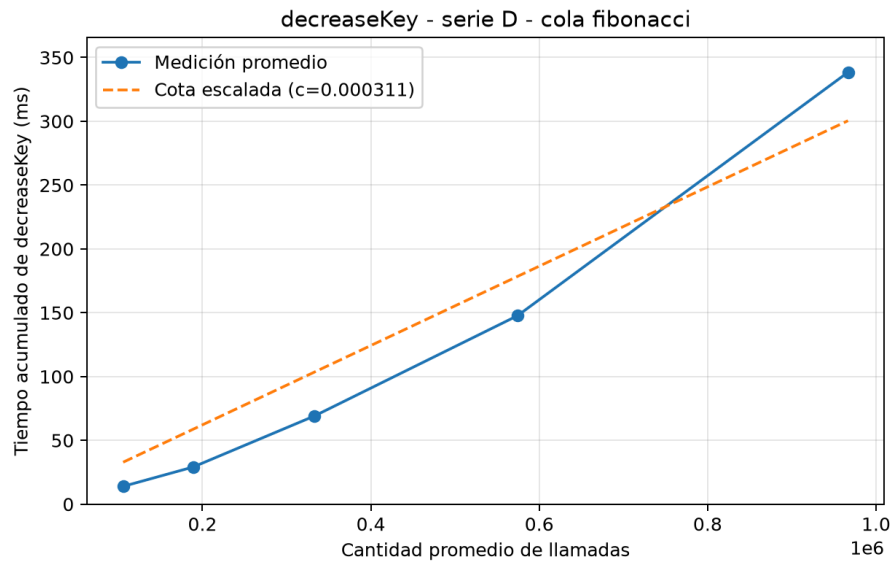


Figura 10: Serie D · tiempo acumulado · cola de Fibonacci.

Serie D · operaciones estructurales

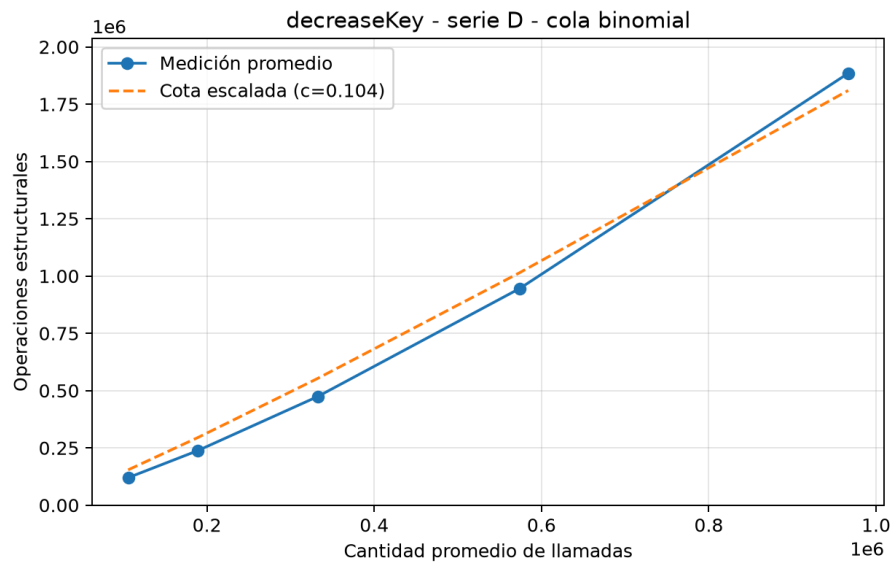


Figura 11: Serie D · operaciones estructurales · cola binomial.

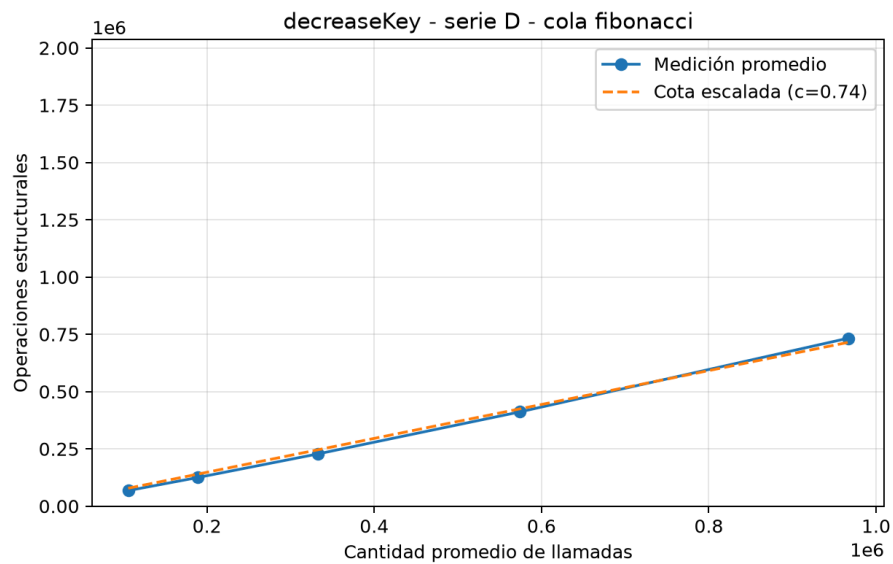


Figura 12: Serie D · operaciones estructurales · cola de Fibonacci.

7.3. Interpretación y constantes

Para mediciones y_i y valores teóricos f_i , el ajuste por mínimos cuadrados sin intercepto es

$$\alpha = \frac{\sum_i f_i y_i}{\sum_i f_i^2}.$$

Se usan $f = e \log_2 v$ y $f = e + v \log_2 v$ para costo total, y $f = k \log_2 v$ y $f = k$ para las llamadas a `decreaseKey`, donde k es su cantidad efectiva. Estas constantes dependen de las unidades, del modelo y de la máquina y no son comparables como si representaran el mismo costo elemental.

En A, e aumenta 16 veces pero el tiempo crece solo aproximadamente 2.10 veces con binomial y 2.38 con Fibonacci. El término fijo asociado a un millón de vértices y el hecho de que solo una fracción de las aristas produce mejoras ayudan a explicar por qué una cota de peor caso no predice exactamente los tiempos. En B, e permanece fijo y $\log_2(v)$ solo pasa de 18 a 22, mientras el tiempo crece más de 12 veces. El trabajo dependiente de v , la cantidad efectiva de mejoras y posibles efectos de memoria quedan ocultos en la cota simplificada $O(e \log v)$.

En C, los intercambios binomiales disminuyen inicialmente de 1762728 a 1609226 pese a que aumentan las llamadas. Esto no contradice $O(k \log v)$, se puede decir que la cota limita el costo superior, no exige un número constante de intercambios por llamada. Al aumentar las alternativas del grafo puede cambiar el estado de la cola y la profundidad recorrida. Los cortes de Fibonacci crecen más suavemente y su conteo por llamada baja de aproximadamente 1.35 a 0.76. Ambas observaciones son compatibles con amortización, sin constituir una prueba experimental de la cota.

Los gráficos C y D representan costos agregados de llamadas intercaladas con `extractMin`. El potencial generado por otras operaciones también participa en el análisis del algoritmo completo, un ajuste lineal aislado de cortes no reemplaza la demostración teórica. El costo del reloj, la asignación de memoria, la caché y la instrumentación pueden afectar los tiempos. Para separar sus efectos se requieren mediciones adicionales.

8. Aplicación

Una aplicación directa es diseñar una red física de costo mínimo entre centros de distribución. Se busca conectar todas las instalaciones mediante enlaces físicos bidireccionales minimizando el costo total de construcción, sin exigir redundancia ni rutas mínimas entre cada par. Si se necesitan tolerancia a fallos o rutas de transporte, un MST por sí solo no satisface esos objetivos. Los vértices representan instalaciones, las aristas enlaces posibles y los pesos el costo de construcción. Si el grafo de alternativas es denso, Prim realiza muchas disminuciones de clave y la cota amortizada de la cola de Fibonacci puede ser útil. Si el grafo es pequeño o las constantes dominan, la cola binomial puede resultar más simple y competitiva.

9. Conclusión

Se implementaron completamente el generador, Prim, la cola binomial y la cola de Fibonacci, incluyendo handles directos, marcas y cortes en cascada. Las 200 comparaciones experimentales reportadas entregan evidencia de consistencia entre implementaciones. La igualdad de peso y las $v-1$ aristas no demuestran por sí solas lo más óptimo así que conviene complementarlas con pruebas pequeñas de respuesta conocida o con un algoritmo independiente.

La hipótesis se cumple parcialmente. Fibonacci evidencia la ventaja amortizada en el conteo de operaciones y llega a superar levemente a binomial en los dos casos con mayor cantidad de vértices de B. Sin embargo, binomial fue más rápida en la mayoría de las configuraciones con constantes y efectos de memoria como posibles explicaciones. Como trabajo futuro sería útil separar el costo del reloj mediante mediciones por lotes y repetir la corrida en Linux para estudiar el efecto del asignador, la caché y el compilador.